

Objective

To implement and demonstrate NumberFormatException in Java using try-catch by attempting to convert a non-numeric string into an integer.

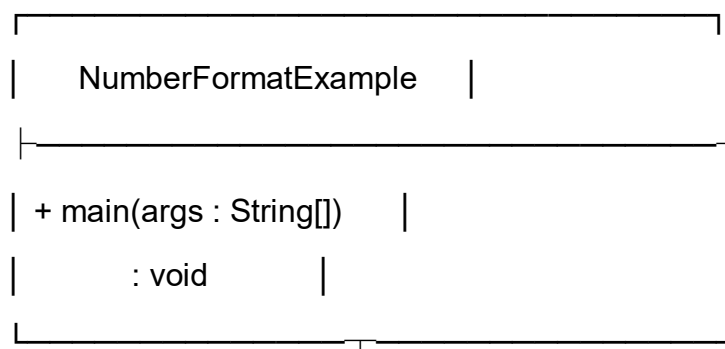
Steps for Execution

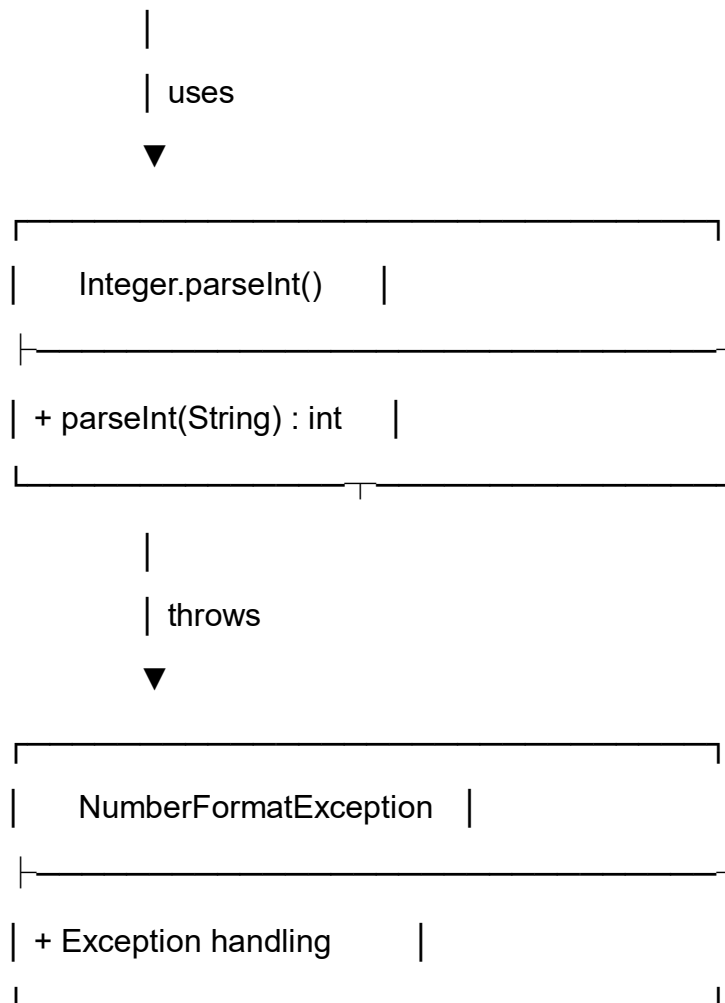
1. Create a Java project and package named javacore.
2. Create a file named NumberFormatExample.java and enter the given code.
3. Declare a String variable with the value "twenty".
4. Try to convert the String into an integer.
5. Handle the exception using catch.
6. Compile and run the program.
7. Observe the output.

Algorithm

1. Start the program.
2. Store "twenty" in a String variable.
3. Try to convert the String into an integer.
4. Catch NumberFormatException.
5. Display the error message.
6. Display "Program ended.".
7. Stop the program.

UML Class Diagram





Relationship

- `NumberFormatException` uses `Integer.parseInt()` to convert the `String` into an integer.
- The value "twenty" is not a valid numeric string.
- `NumberFormatException` occurs and is handled using the catch block.

CODE:

```
package javacore;

public class NumberFormatExample {

    public static void main(String[] args) {

        String age = "twenty";

        try {
```

```
        System.out.println("Age value: " + age);

        int number = Integer.parseInt(age);

        System.out.println("Age: " + number);
    }
    catch (NumberFormatException e) {
        System.out.println("Error: The given value is not a number.");
    }

    System.out.println("Program ended.");
}
}
```

Output

Age value: twenty

Error: The given value is not a number.

Program ended.

Result

The program successfully demonstrates `NumberFormatException` handling in Java. Since "twenty" is not a valid numeric string, `Integer.parseInt()` throws a `NumberFormatException`, which is handled using the catch block.